チャットを送る

ボットアカウント名で配信にチャットを送ります。Spoon アプリのチャット欄にはボットアカウントの ニックネームで表示されます。

メソッド・パス	`POST /v1/live/chat`
必要な権限	`chat.send`
成功レスポンス	`204`(ボディなし)
配信中でないとき	`404`(`OAPI_MNGR_0301`)

```
export SPOON_BASE_URL="https://jp-openapi.spooncast.net"
export SPOON_ACCESS_TOKEN="{発行された access token}"
```

韓国リージョンなら `https://kr-openapi.spooncast.net` です — リージョン別アドレス。

リクエスト

```
POST /v1/live/chat
```

ヘッダー

名前	必須	説明
`Authorization`	✔	`Bearer {access_token}` 形式です。
`Content-Type`	✔	`application/json` です。

リクエストボディ

```
{
  "message": "こんにちは！ボットからのごあいさつです"
}
```

フィールド	型	null	説明
`message`	string	いいえ	送るチャット本文です。200 文字までです。空または空白のみだと `400` です。

どの配信に送るかは **トークンが決めます**。パスにもボディにも配信を指す値はありません。

リクエスト例

```
curl -i -X POST "$SPOON_BASE_URL/v1/live/chat" \
  -H "Authorization: Bearer $SPOON_ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"message":"こんにちは！ボットからのごあいさつです"}'
```

レスポンス

成功すると **ボディなしで** `204` です。パースするものがないので、ステータスコードだけを見ます。

```
HTTP/2 204
```

コード

```
// send.ts — node --experimental-strip-types send.ts "送るメッセージ"

interface ApiError {
  detailCode: string;
  message: string;
}

const res = await fetch(`${process.env.SPOON_BASE_URL}/v1/live/chat`, {
  method: 'POST',
  headers: {
```

```
    Authorization: `Bearer ${process.env.SPOON_ACCESS_TOKEN}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ message: process.argv[2] }),
});

if (res.status === 204) {
  console.log('送りました。');
} else if (res.status === 404) {
  console.log('配信中ではありません。');
} else if (res.status === 429) {
  console.log('今はチャットが混み合っています。しばらくして送り直します。');
} else if (res.status === 401) {
  console.error('連携が切れました。DJ に再連携を案内します。');
} else {
  // 権限不足の 403 は手前で止められるため detailCode がありません
  const { detailCode, message } = (await res.json()) as Partial<ApiError>;
  console.error(`送信失敗 ${res.status} ${detailCode ?? '権限不足'}: ${message
?? ''}`);
}
```

```
node --experimental-strip-types send.ts "こんにちは"
```

注意

送ったメッセージは自分のストリームには戻りません。イベントストリーム はボット自身の発言をそのボットに 返しません。自分のあいさつをコマンドとして再実行する事故を防ぐためです。同じ配信の 他のボットの発言は見えます。

200 文字を超えると `400` です。文字数は UTF-16 code unit 基準なので、絵文字ひとつが 2 文字として数えられることがあります。

`429` **はボットが送りすぎたときだけではありません。**送信の上限を配信全体で分け合うため、リスナーが活発に発言していると、ボットが何もしていなくてもかかります。ボットの速度を落として もすぐには解消しないので、少し待ってから送り直します。

DJ が止めればボットも止まります。チャットを凍結したり(`isChatFrozen`)ボットをチャット 禁止にすると `403`、強制退出させると入場自体ができません。発言の前に 現在の配信を取得 で `isChatFrozen` を確認すれば、失敗を先に避けられます。

ストリームに接続しなくても送れます。部屋に入っていなければサーバーが先に入場させます。

エラー

HTTP	detailCode	何が起きたか	どうするか
400	OAPI_MNGR_0103	リクエストが正しくありません。ボディが壊れているか、必須フィールド・Content-Type が抜けているか、メソッドが誤っています。	リクエストの形式を直します。同じリクエストを送り直しても結果は同じです。
400	OAPI_MNGR_0108	message が空、または空白のみです。	送る内容を確認します。
400	OAPI_MNGR_0109	200 文字を超えました。	長さを短くします。
401	なし	トークンがないか無効です。	DJ に再連携を案内します。
403	なし	chat.send の権限がありません。	権限を追加して再同意を得ます。
403	OAPI_MNGR_0208	チャットが凍結されているか、ボットがチャット禁止の状態です。	2 つは区別できません。少し後に再試行し、それでも通らなければ DJ に案内します。
403	OAPI_MNGR_0209	配信に入場できません。DJ がボットをブロックした可能性があります。	再試行では解決しません。DJ が解除する必要があります。
404	OAPI_MNGR_0301	配信中ではありません。	配信が始まるまで待ちます。
429	OAPI_MNGR_0302	チャットの送信が頻繁すぎます。	少し待ってから送り直します。
502	OAPI_MNGR_2302	配信サーバーのエラーです。送信前の配信確認・入場の段階で起きます。	バックオフしてから再試行します。

HTTP	`detailCode`	何が起きたか	どうするか
`502`	`OAPI_MNGR_2303`	チャットサーバーのエラーです。	バックオフしてから再試行します。

`429` には 2 種類あります。上の `OAPI_MNGR_0302`(チャット送信の上限)にはボディが ありますが、API の呼び出し上限にかかった場合は **ボディも `detailCode` もありません。**ボディをパースする前にステータスコードで分けないと落ちます — 上のサンプルはそう書かれています。

この表にない状態も返ることがあります。誤ったメソッド・`Content-Type`(`400`・`405`・`415`、`OAPI_MNGR_0103`)・呼び出し上限(`429`、ボディなし)・サーバーエラー(`500`、`OAPI_MNGR_1002`)は **すべてのエンドポイントに共通** です — エラーコード にあります。

関連ドキュメント

- エンドポイント一覧 — エンドポイントの全一覧です。
- 現在の配信を取得 — 発言の前に `isChatFrozen` を確認します。
- イベントストリーム — リスナーの反応を受け取って返します。
- エラーコード — エラーコードの一覧です。